CodeMind

Technologies used, why we used them, and what we built new

CodeMind is an AI codebase assistant. You connect a GitHub repository; it reads and indexes the code, answers plain-English questions about it with sources, and checks each new commit for breaking changes. By default every AI step runs **locally** on the developer's own machine, so no source code leaves it.

1. AI and retrieval

Technology	What it is (full definition)	Why we used it
RAG	**Retrieval-Augmented Generation.** Before the AI model answers, the system first searches the repository for the most relevant code and gives that code to the model together with the question.	Small models do not know your code. RAG makes answers grounded in it, with cited files, and lets the app say "not enough information" instead of guessing.
LangChain	An open-source framework for building applications around language models. It provides standard pieces (prompt templates, model wrappers, output parsers) that can be chained into a pipeline.	Our RAG answer is a LangChain chain: prompt, then model, then output parser. It keeps retrieval and generation separate and swappable. This is an A2 rubric item.
Ollama	A program that downloads and runs open-source language models on your own computer and serves them through a local web API.	Runs the AI privately and free of cost, with no API key and no code sent to the cloud.
qwen2.5-coder: 3b-instruct	A 3-billion-parameter language model from Alibaba's Qwen family, specialised for code and tuned to follow instructions.	Small enough to fit a 4 GB GPU, fast, and gives direct answers. We tested Qwen3 models and rejected them because they wasted tokens on hidden reasoning.
nomic-embed-text	An embedding model that turns text into a list of 768 numbers (a vector) so that texts with similar meaning get similar numbers.	Used to make code and questions comparable by meaning. Embeddings always stay local.
Embeddings	A vector (list of numbers) that represents the meaning of a piece of text or code.	They let us find code by meaning, not only by matching words.
Vector search	Finding the stored vectors closest to a query vector, usually with cosine similarity (the angle between two vectors).	This is how a question like "where is login handled?" finds the right files.
PostgreSQL + pgvector	**PostgreSQL** is an open-source relational database. **pgvector** is an extension that adds a vector column type and similarity search to it.	One database holds projects, commits and vectors, so every search can be filtered by repository (no cross-repository leaks).
Qdrant	A dedicated open-source vector database built for fast similarity search at large scale.	Provisioned in Docker as the future main vector store. Today pgvector does the retrieval.
Google Gemini, Groq	Cloud AI services. Gemini is Google's model family; Groq serves open models (such as GPT-OSS) on very fast hardware.	Optional. A model switcher lets the user compare a local model against a cloud model. Used only when an API key is set and the user chooses it.

2. Application

Technology	What it is (full definition)	Why we used it
TypeScript	JavaScript with static types, checked before the code runs.	Catches mistakes early across the whole codebase (zero type errors).
Next.js 15 and React 19	**React** is a library for building user interfaces from components. **Next.js** is a framework on top of it that adds routing, server rendering and API routes.	One project serves both the web pages and the backend API (including the webhook).
tRPC	**TypeScript Remote Procedure Call.** Lets the frontend call backend functions as if they were local, with types shared end to end and no hand-written API layer.	Fewer bugs between browser and server, and fast development.
Prisma	An **ORM** (Object-Relational Mapper): you describe tables in a schema file and use typed code instead of raw SQL.	Safe, typed database access. Vector queries use raw SQL where needed.
Zod	A library that checks data against a schema at runtime.	Validates input and environment settings, so bad data is rejected early.
Tailwind CSS, Radix UI, shadcn/ui, Recharts	Tailwind is utility-class styling. Radix provides accessible UI building blocks. shadcn/ui is a component set built on them. Recharts draws charts.	A clean, accessible dashboard with charts for trends and metrics.
Octokit	GitHub's official library for its REST API.	Fetches repository files, commits and diffs.

3. Testing, automation and operations

Technology	What it is (full definition)	Why we used it
Vitest	A fast test runner for TypeScript and JavaScript.	Runs 137 unit tests (AI mocked) and integration tests against a real pgvector database.
Docker and Docker Compose	**Docker** packages an app and its dependencies into a portable container. **Compose** starts several containers together from one file.	One command starts the app, PostgreSQL and Qdrant the same way on every machine.
GitHub Actions (CI/CD)	**Continuous Integration / Continuous Delivery.** Automated workflows that run on GitHub whenever code changes: test it, build it, publish it.	Every push is linted, type-checked, tested and built. The Docker image is published automatically. This is an A2 rubric item.
GHCR	**GitHub Container Registry.** A place to store and share Docker images.	The CD workflow publishes the production image here.
Sweep.dev	An AI agent that turns a GitHub issue into a draft pull request.	A2 rubric item. Configured in sweep.yaml; needs its GitHub App installed.
Qodo (formerly CodiumAI)	An AI tool that reviews code and generates tests. Its open-source PR-Agent reviews pull requests.	A2 rubric item for AI test generation and automated review.
Prometheus and Grafana	**Prometheus** collects numeric metrics over time. **Grafana** draws dashboards from them.	Shows live latency, token use and error counts for the AI pipeline.

4. Ideas used in the new work

Idea	What it is (full definition)	Why we used it
Hybrid retrieval	Running a meaning-based (vector) search and a keyword search together, then merging the results.	Vector search can blur exact names such as *verifyGithubSignature*; keyword search finds them.
Reciprocal Rank Fusion	A way to merge ranked lists: each item scores 1 / (60 + its rank) in every list it appears in, and the scores are added.	Needs no score normalising, and items both searches agree on rise to the top.
Chunking	Splitting a file into smaller pieces (here, function by function) before embedding.	Finds the exact function instead of a whole file, which fits small context windows.
Evaluation metrics	**Hit rate**: share of questions with a correct file in the top results. **MRR** (Mean Reciprocal Rank): average of 1/rank of the first correct result. **Precision**: share of results that are correct.	Turns "retrieval seems better" into numbers.
Webhook and HMAC	A **webhook** is a web call GitHub makes when something happens. **HMAC** (Hash-based Message Authentication Code) is a signature made with a shared secret.	Lets GitHub notify CodeMind on every push, and lets CodeMind reject forged requests.

5. What we built new

CodeMind is built on **git-inspect** by Parth Pankaj Singh (MIT licence). That project supplied the Next.js and tRPC base, GitHub loading, commit analysis and the dashboard, powered by Google Gemini. Everything below is new work in CodeMind.

Feature	What was built
Local-first AI	Ollama is the default behind LLMProvider and EmbeddingProvider interfaces. Gemini and Groq are optional and switchable.
LangChain RAG pipeline	Retrieval separated from generation, a context budget for small models, honest "not enough information" replies, relevance scores, and a no-retrieval baseline to compare against.
RAG on / off / compare	Each question can be answered with retrieval, without it, or side by side, to show what retrieval adds.
Function-level index	Files are split into function chunks with line ranges, and answers cite the exact functions.
Auto-scale answers	Escalation to a larger model, with user approval.
Model evaluation	Compared three local models on constrained hardware and chose the best fit.
Connect and status pages	Preview of which files will be indexed, live indexing progress, and a System Status page for Ollama, models, PostgreSQL and Qdrant.
Single local user	Removed the third-party sign-in service. Every search stays scoped to one repository.
Bug fixes	URL parsing, diff summaries, breaking-change JSON parsing and the sign-in redirect.
Tests	137 unit tests and 8 integration tests on real pgvector, including a check that one repository can never read another's code.
DevOps	Multi-stage non-root Docker image, Docker Compose, GitHub Actions CI, CD to GHCR, and an AI pull-request review that runs a small local model in the runner.
Observability	Prometheus metrics and a Grafana dashboard.

Feature	What was built
New for A2: hybrid retrieval	Vector plus PostgreSQL full-text search merged with Reciprocal Rank Fusion. Switch on with RETRIEVAL_MODE=hybrid.
New for A2: evaluation lab	npm run eval compares vector and hybrid retrieval on a fixed question set and reports hit rate, MRR and precision.
New for A2: GitHub webhook	A signed push event makes CodeMind analyse new commits automatically.
New for A2: Sweep and Qodo setup	sweep.yaml, a Sweep issue template, and a Qodo PR-Agent workflow.

Status note. Sweep.dev and Qodo are configured in the repository, but they only run once their accounts or GitHub App are connected. The hybrid retrieval, evaluation lab and webhook are implemented and tested; run npm run eval on an indexed repository to get the comparison numbers.